

WEEK 06 · TAILWIND CSS · UTILITIES · @THEME · COMPONENTS

THE PAGE WRITTEN IN CLASSES

Same CSS. Different place to put it. And the first week Claude writes code with you.

WHERE WE ARE

Week 6 of 12. You can build any layout — and last week it took a stylesheet of hundreds of lines.

THE NEXT 180 MINUTES

Five cycles, you typing in every one: one card, translated · the shell in twelve columns · one link, four states, a second ramp · three answers to one prompt · one button, six copies. Break at 90, assignment at 145.

WHAT THIS UNLOCKS

Every page from here, the project included — from a file beside the page, with the network off.

TARGET · MINUTE 0

THIS IS WHERE YOU WILL BE AT MINUTE 180

the class assignment — last week's dashboard, and no stylesheet at all

reload

open

Signal

SERVICES

Gateway

Checkout

Search

Mailer

Reports

Gateway

Everything on this page is one grid holding five regions, and one grid inside it holding a row of cards that nobody counted in advance.

Requests

1.42M

Across all regions in the last twenty-four hours, including the retries that the gateway absorbed on the client's behalf.

updated 2 min ago

Latency

86ms

Errors

0.31%

Signal

Overview Activity Components

Gateway

Activity

targets/week-06.png · 1280

Signal

Overview Activity Components

Gateway

Everything on this page is one grid holding five regions, and one grid inside it holding a row of cards that nobody counted in advance.

► **THE POINT** — the markup is last week's, element for element, and there is no stylesheet. Every class is a declaration you already know: a twelve-column shell, cards on the scale, a focus ring, a dark ramp that passes 4.5:1 in both schemes, one definition per component. You type every piece of it today: once in a cycle, once in the assignment.

AI THIS WEEK

ASSISTANT. IT WRITES; YOU CHOOSE, AND SAY WHY IN WRITING.

LAYER · WEEKS 6-7 · ASSISTANT

ALLOWED

Snippets, class lists, three versions of one component. Run every one where your project runs — `vendor/tailwind.js`, no network, 375 and 1280, Tab. Keep one, and write down what you rejected and why.

NOT ALLOWED

A line you cannot explain. A whole page. Anything off the course's stack — or off its setup: a CDN, a `tailwind.config`. Keeping output without a written reason.

Cycle 1's you-do is AI-free, and so is cycle 3's.

NO SUBSCRIPTION?

Everything works in the free chat, and nothing in the course depends on a paid plan. Cycle 4 is one prompt in three chats — whatever chat you have.

CYCLE 1 · 25 MIN

ONE CARD, TRANSLATED

A card written in ordinary CSS becomes classes, declaration by declaration, until its `<style>` block is empty and deleted — and the card has not moved. On the way: the reset Tailwind brings with it, `@theme` as week 3's tokens, and the one attribute that decides whether a token exists at all.

1 I DO

10 MIN

3 on the next three slides, 7 in the editor · live-code/01-card-translated · the script line · `@theme` in the wrong block, then in its own · the card, the text, the button · the plain `<style>` deleted

2 WE DO

5 MIN

you type it on your copy, the tokens straight into `type="text/tailwindcss"` · compare with checkpoint 1

3 YOU DO

8 MIN

warmup/ — match the box until the judge says 0 · demos/03-translator, and divide by four

4 DEBRIEF

2 MIN

a class with no rule is silent · p-[16px] against p-4 · ↗ reference

THE SAME CARD, WITH NO STYLESHEET AT ALL

```
1 <div class="flex items-start gap-3 rounded-xl border border-slate-300 bg-white p-4">
2   <div class="grid h-8 w-8 flex-none place-items-center rounded-full bg-blue-100">!</div>
3   <div class="min-w-0">
4     <p class="mb-0.5 font-semibold">Certificate expires in 6 days</p>
5     <p class="text-sm text-slate-600">Renew it before Friday.</p>
6   </div>
7 </div>
```

DEAD RULES: STRUCTURALLY ZERO

Not "fewer". There is no stylesheet, so there is nothing that **can** die.

NAMES INVENTED: ZERO

Nothing to remember, nothing to defend in review, nothing to rename.

SPECIFICITY: 0,1,0. ALWAYS.

Every utility is exactly one class. There is nothing to escalate.

@theme IS tokens.css WITH A SECOND JOB

WEEK 3 - :root, YOUR TOKENS

```
:root {  
  --color-accent: hsl(234 89% 52%);  
}  
  
.brand { color: var(--color-accent) }
```

WEEK 6 - THE SAME VALUES, IN <STYLE TYPE="TEXT/TAILWINDCSS">

```
@theme {  
  --color-brand-500: hsl(234 89% 52%);  
}  
  
/* and now: text-brand-500,  
   bg-brand-500, border-brand-500 ... */
```

A **@theme** entry is a custom property,
and also generates a family of classes.

THE TEST, AND IT IS ONE QUESTION

Would you want a second element to have this exact value?

NO → BRACKETS

A 728×90 ad slot. A `mask-image` used once in a career. Nobody chose it and nobody will reuse it.

YES → `@theme`

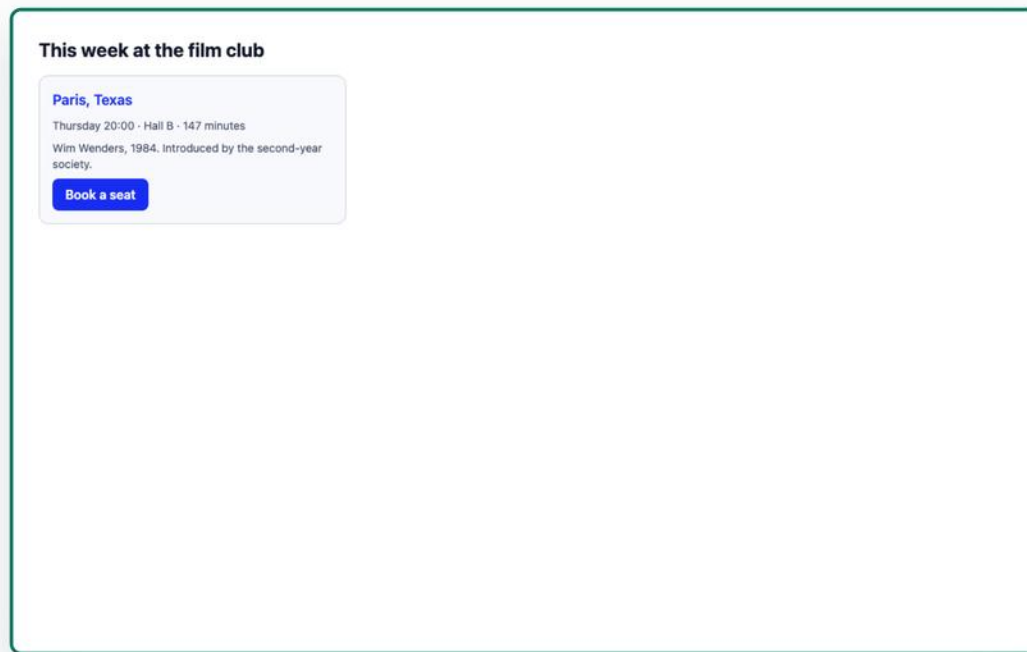
It is a decision. Give it a name, and it becomes a first-class utility.

ALREADY ON THE SCALE → THE SMELL

`p-[13px]` when `p-3` exists. You are overriding a system that was working, one pixel at a time.

CHECKPOINT 1

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/01-card-translated/checkpoint.png` is this picture at 1280; `live-code/01-card-translated/end/` is the code behind it. The card 384 wide, the title in the brand colour, and one `<style>` left in the page — the one with `type="text/tailwindcss"`.

CYCLE 2 · 25 MIN

THE SHELL, IN TWELVE COLUMNS

Five regions, three arrangements, and not one media query in the file: one column on a phone, two at 768, three at 1280. Then the one utility that puts the main column first on a phone — and puts it before the header too, until you read what "first" means.

1 I DO

10 MIN

3 on the next three slides, 7 in the editor · live-code/02-shell-twelve-columns · grid-cols-1 · md:grid-cols-12 and the spans · xl: · order-first, and what it does to the header · md:order-none · Tab at 375 · flex-1 on the stats

2 WE DO

5 MIN

you type it on your copy, order-first on both regions from the start · compare with checkpoint 2 at 1280, then drag to 375

3 YOU DO

8 MIN

one more width, lg:, and a region that moves · AI as assistant: a class list is allowed, a class you cannot explain is not

4 DEBRIEF

2 MIN

first of every item · Tab follows the DOM · flex-1 already means basis 0 · ↗ reference

YOU DO · AI AS ASSISTANT

↗ REFERENCE · 19, 27–29

YOU WROTE THIS LAST WEEK. IT IS THE SAME RULE.

WEEK 5 — BY HAND

```
.kpis {  
  display: grid;  
  grid-template-columns: 1fr;  
}  
  
@media (min-width: 48rem) {  
  .kpis {  
    grid-template-columns: repeat(3, 1fr);  
  }  
}
```

WEEK 6 — AND IT GENERATES EXACTLY THAT

```
<section class="grid grid-cols-1  
             md:grid-cols-3">
```

Nine lines and an invented class name become one attribute. The **breakpoint did not move** and the CSS is identical — you can read it back out of the browser and check.

AND THE ARITHMETIC TRANSFERS UNCHANGED

LESSON 6, BOOTSTRAP

```
<div class="row">
  <div class="col-lg-3">...</div>
  <div class="col-lg-6">...</div>
  <div class="col-lg-3">...</div>
</div>
```

WEEK 6, AND THE ASSIGNMENT'S SHELL

```
<div class="grid md:grid-cols-12">
  <nav class="md:col-span-4 xl:col-span-2">
  <main class="md:col-span-8 xl:col-span-7">
  <aside class="md:col-span-12 xl:col-span-3">
</div>
```

Twelve columns, because twelve divides by 2, 3, 4 and 6. Every design tool in the industry speaks it.

THERE IS NO `max-*` BY DEFAULT, AND THAT IS A DECISION

A MAX-WIDTH QUERY

States what a thing **stops** being.
Every one has to be undone by the next size up — which is the tangle last week's assignment made you untangle.

A PREFIX

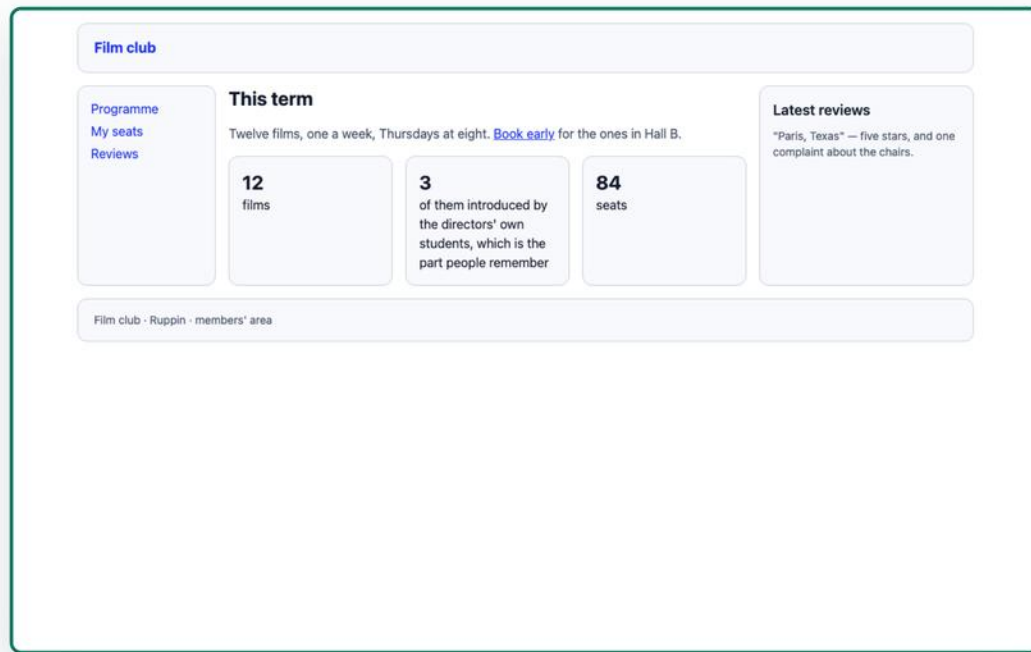
Only ever **adds**. Four of them can match at once and none of them fights. Mobile-first is not a discipline here; it is the shape of the tool.

THE ESCAPE HATCH EXISTS

`max-md:` is real, because a system with no escape hatch is one you abandon. It is still the wrong default. **Remember that it exists.**

CHECKPOINT 2

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/02-shell-twelve-columns/checkpoint.png` is this picture at 1280; `live-code/02-shell-twelve-columns/end/` is the code behind it. Three columns at 1280 and three equal tiles (205 each); at 375 the header, then the content, then the menu. No `@media` and no `max-` in the file.

CYCLE 3 · 30 MIN

ONE LINK, FOUR STATES, AND A SECOND RAMP

A link and a button get every state the assignment checks: a hover, a ring that appears only from the keyboard, a transition that does not exist for someone who asked for less motion, a disabled look. Then dark mode — and the brand colour that was fine in light is 2.36:1 in dark.

1 I DO

10 MIN

3 on the next two slides, 7 in the editor · live-code/03-link-four-states · hover: · focus-visible:outline-2 · motion-safe:transition-colors · disabled: · dark: as an inversion, then as a second ramp

2 WE DO

8 MIN

you type it on your copy, brand-300 in dark from the start · Tab with the mouse down, Rendering in dark and with reduce · compare with checkpoint 3

3 YOU DO

10 MIN

a fourth link and a card hover, in both schemes · the colour picker, Tab, Rendering

4 DEBRIEF

2 MIN

focus: against focus-visible: · 2.36 against 7.22 · ↗ reference

YOU DO · AI-FREE

➤ REFERENCE · 10, 31, 35

FIVE STATES, AND NO JAVASCRIPT IN ANY OF THEM

hover · focus-visible · active · group-hover · peer

reload open

hover: idle

point at me

```
@media (hover: hover)
.hover\:bg-brand-700:hover {
  background-color: var(--color-brand-700);
}
```

Notice the media query. This is why hover never sticks on a phone.

focus-visible: idle

reach me with Tab

```
.focus-visible\:outline-brand-500:focus-visible
{
  outline-color: var(--color-brand-500);
}
```

The one you cannot demonstrate with a mouse, and the one always graded.

active: idle

hold the button down

```
.active\:bg-brand-700:active {
  background-color: var(--color-brand-700);
}
```

Lasts exactly as long as the mouse button is down. Rarely enough on its own.

group-hover: idle

Gateway
hover anywhere on the card — this line changes

```
@media (hover: hover)
.group-hover\:text-brand-500:is(:where(.group):hover *) {
  color: var(--color-brand-500);
}
```

Parent hovered, child styled. Zero JavaScript — read the selector.

peer-checked: idle

☐ email me when a deploy fails

```
.peer-checked\:text-brand-500:is(:where(.peer):checked ~ *) {
  color: var(--color-brand-500);
}
```

► **HANDS-ON** [demos/06-variants/](#) — do the first three with the mouse, then **PUT THE MOUSE DOWN** and reach the fourth with Tab.

A DARK RAMP IS A SECOND SET OF DECISIONS, NOT AN INVERSION

WHAT PEOPLE EXPECT

Flip the background, flip the text, ship it.

WHAT ACTUALLY HAPPENS

`brand-500` on near-black is **2.36:1**
— measured in cycle 3. Under 3 for a ring, under 4.5 for a word.

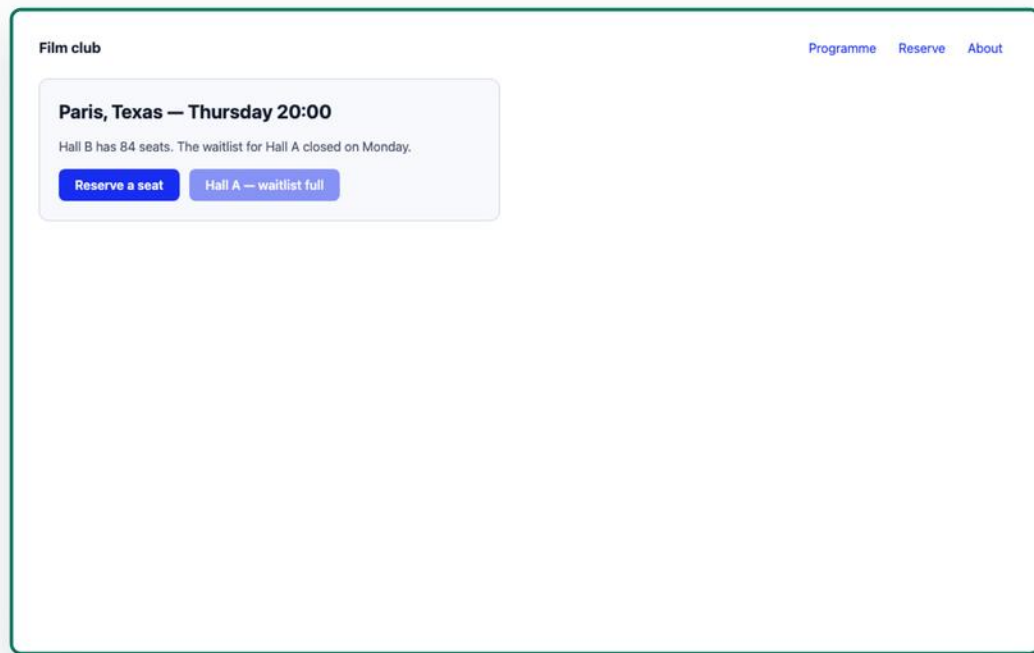
WHAT YOU DO

A lighter cut of the same colour.
4.5:1 in both ramps, independently.

And no toggle: `dark:` is a media query on a preference the user already set.

CHECKPOINT 3

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/03-link-four-states/checkpoint.png` is this picture at 1280; `live-code/03-link-four-states/end/` is the code behind it. Tab with the mouse down: a solid 2px ring on every link. Rendering in dark: the links in light indigo, 7.22:1. With `prefers-reduced-motion: reduce`: `transition-duration: 0s`.

BREAK

10 MINUTES

Back at 100. Cycle 4 is the AI cycle — have a chat window open, free or paid, Live Server running, and

`ai-lab/04-three-answers-one-prompt/artifact/start/` in a tab.

CYCLE 4 · 25 MIN

THREE ANSWERS TO ONE PROMPT

One naive request — "Tailwind from a CDN, responsive, in my brand colour" — sent in three chats. Three answers, all well explained. Run them the way your project runs: one of them keeps its colour in a file this week's Tailwind never reads. The other two both work, and choosing between them is what you write down.

1 I DO

7 MIN

2 on this slide, 5 in the chat · ai-lab/04-three-answers-one-prompt · P1 in three chats, live · run each in artifact/start/, vendor/tailwind.js, no CDN · transcript.md if the room got something else

2 WE DO

6 MIN

everyone sends P1 · run what came back at 375, 1280, with Tab and the console · find the one that breaks, and under which condition, before it is named

3 YOU DO

10 MIN

write P2 as checkable constraints · run it offline · the review exemplar on screen · then the three parts: chosen, broken, the correct one rejected

4 DEBRIEF

2 MIN

right for another setup is wrong for yours · ↗ the review exemplar

CYCLE 4 · BEFORE YOU WRITE

WHAT A REVIEW LOOKS LIKE, BEFORE YOU WRITE ONE

AI-REVIEW-GOOD.MD

Claim and conclusion apart. "The explanation says two columns below 1024. *I checked at 700:* three columns, 217px each." Every sentence names the width, the number, the tool. The longest part is why the **correct** version was rejected — and when it would have been chosen.

AI-REVIEW-BAD.MD

Same three versions. "It looked cleanest." "I saw it in the code." "Grid is not allowed this week." True, short, and worth nothing: no width, no measurement, no cost, no "when would I pick it".

Three parts: what you chose and why · what is broken, how you found it, under which condition · the correct one you rejected, with something in the code and what it costs.

CHECKPOINT 4

THREE QUESTIONS BEFORE THE NEXT CYCLE

1. Did you **run** every answer from `vendor/tailwind.js`, with the CDN line left out — and not stop because it looked right with the network on?
2. Is every constraint in your P2 something the browser can check — where Tailwind loads from, where the colour lives, `md:` and no `max-*`, `focus-visible:`, `motion-safe:` — and does what came back meet all of them offline?
3. Does your part on "the correct one I rejected" name **something in the code and what it costs** — "the brand colour written seven times", not "less clean"?

Compare — `ai-lab/04-three-answers-one-prompt/checkpoint-he.md` has the same three questions and the measurements; `artifact/answer-a/` to `answer-p2/` are the four answers, ready to open side by side.

CYCLE 5 · 20 MIN

ONE BUTTON, SIX COPIES

A button that appears in six places is defined once, in a gallery at the top of the page, and every copy is pasted from there — because copies typed from memory drift, and the worst drift looks perfect until somebody touches it.

1 I DO

7 MIN

2 on the next two slides, 5 in the editor · live-code/05-one-button-six-copies · find the two drifted copies, by eye then by measuring · paste from the gallery · a sixth typed from memory · paste again · rounded-full, seven times

2 WE DO

5 MIN

you paste, never type, the sixth one too · the console line, then hover and Tab on every button · compare with checkpoint 5

3 YOU DO

6 MIN

the screening card as a component: defined in the gallery, pasted six times · AI as assistant, checked in the console

4 DEBRIEF

2 MIN

replace, do not repair · what @apply solves and what it leaves · ↗ reference

YOU DO · AI AS ASSISTANT

➤ REFERENCE · 38–42

MISCONCEPTION 5

"CLASS SOUP IS JUST WHAT THIS LOOKS LIKE."

six copies of one button, and what happened to them

↶ reload

open ↷

TODAY SIX COPIES

The same button, typed out six times. Find the odd ones out by eye before you read the audit.



```
<button class="cursor-pointer rounded-lg bg-brand-500  
px-5 py-2 text-sm font-semibold text-white  
hover:bg-brand-700">Renew</button>
```

...and five more, each typed out again

6 buttons, meant to be identical.
distinct renderings: 2
carrying a hover rule: 5 of 6
renders differently: Scale
has no hover state: Roll back

Two of the six have already drifted, and neither drift was a decision.
One has px-4 where the rest have px-5 – four pixels, invisible in a row.

THIS WEEK ONE SOURCE

The button is defined once, on `components.html`, and every use is a copy of *that* — by hand, no build step. That is what CA6 asks for. *On this page* the six are generated from one string, so that the demo has a single source too; the discipline is the same and the audit cannot tell them apart, which is the point.



`components.html` – the ONE definition:

```
<!-- button / primary -->  
<button class="cursor-pointer rounded-lg  
bg-brand-500 px-5 py-2 text-sm font-semibold  
text-white hover:bg-brand-700">Label</button>
```

everywhere else: copy that line.
changed your mind? change it there, then re-copy.

6 buttons, meant to be identical.
distinct renderings: 1
carrying a hover rule: 6 of 6

WEEK 10 ONE FUNCTION

Read this one; do not write it yet. It needs template literals and functions, which is week 8, and the render pattern it belongs to is week 10. It is here so you know the ladder has a top.



```
const BUTTON =  
  'cursor-pointer rounded-lg bg-brand-500 ' +  
  'px-5 py-2 text-sm font-semibold text-white ' +  
  'hover:bg-brand-700';  
  
const button = (label) =>  
  `<button type="button" class="${BUTTON}">${label}</button>`;  
  
LABELS.map(button).join('')
```

6 buttons, meant to be identical.
distinct renderings: 1
carrying a hover rule: 6 of 6

One rendering, and now it is not a discipline – it is

► **HANDS-ON** `demos/09-extraction/` — panel one. Ask the room to find the odd button **BY EYE** before reading the audit.

MISCONCEPTION 3

“@apply SOLVES THIS.”

the same four measurements, on an @apply stylesheet

↺ reload

open ↗

THE SAME BUTTONS, WRITTEN WITH @APPLY

Renew Delete Archive

and the same two buttons again, inside .panel — which is a real wrapper in a real page and not a trick:

Renew Delete Archive

```
.btn { @apply cursor-pointer rounded-lg bg-brand-500 px-5 py-2 _ }  
.btn-danger { @apply bg-red-600; }  
  
.panel .btn { @apply bg-brand-500; } /* written later, by someone else */
```

the Delete button, measured in both rows:
outside .panel oklch(0.577 0.245 27.325)
inside .panel oklch(0.577 0.245 27.325)

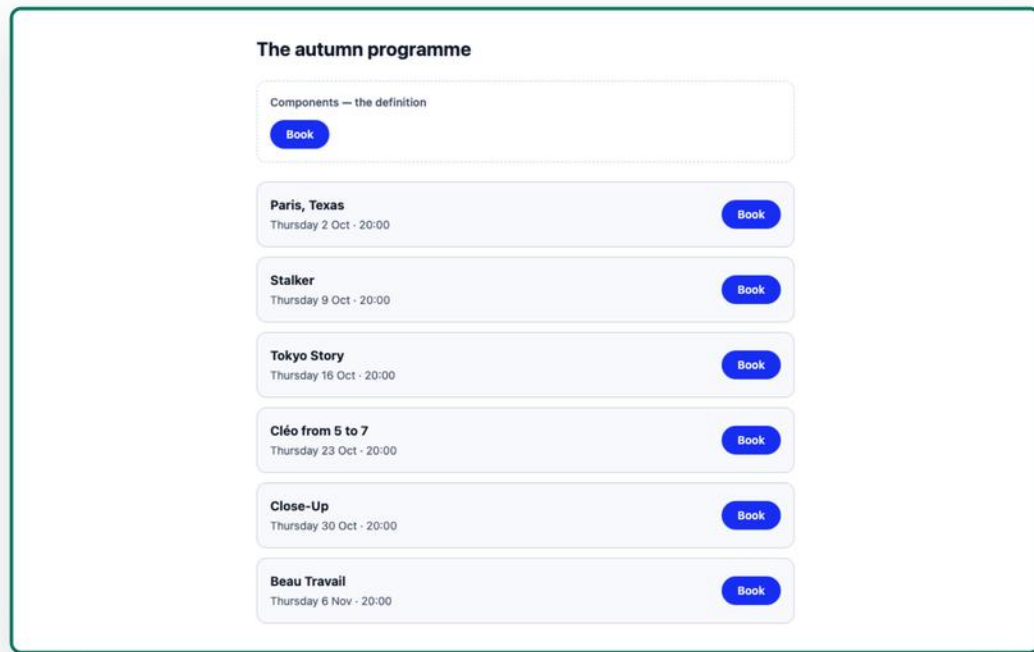
they agree — only because .panel .btn.btn-danger was written to win the fight that .panel .btn started. Delete that one line and the Delete button turns brand-blue inside panels and nowhere else.

Utilities cannot do this to you. `bg-red-600` is one class at (0,1,0) and it

► **HANDS-ON** demos/10-apply-trap/ — write `.btn { @apply ... }` on the board first. Ask whether it beats six copies.

CHECKPOINT 5

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/05-one-button-six-copies/checkpoint.png` is this picture at 1280; `live-code/05-one-button-six-copies/end/` is the code behind it. Six buttons, six hovers, six solid 2px rings on Tab — and every class list on the page pasted from the gallery.

ASSIGNMENT · 145-175

THE ASSIGNMENT: ASSEMBLE WHAT YOU TYPED, THEN EXTEND IT

ASSEMBLE · CORE

Last week's dashboard with no stylesheet: the cards in utilities on the scale (cycle 1) · the shell in twelve columns, the main column before the menu on a phone (cycle 2) · hover and a `focus-visible` ring on the side menu, and a dark ramp that passes 4.5:1 in both schemes (cycle 3) · and the written review of three AI answers (cycle 4).

EXTEND · STRETCH AND CHALLENGE

`components.html`: button, card, badge and field defined once and pasted everywhere (cycle 5) · the broken AI variant fixed in its file · then TinDog, rebuilt with nothing typed from memory.

THE RULES, AND THE EXAM

No stylesheet of your own, no `style=""`, no `@apply`, no **!important.** Tailwind from `vendor/tailwind.js`, never a CDN. AI as assistant: you choose, and say in writing what you rejected. Repo URL and SHA in Moodle. Week 13 asks about this code, offline.

DEBRIEF · 175–180

THREE QUESTIONS BEFORE YOU GO

1. `order-first` on the main column. First of what — and where does Tab go first?
2. Two answers from Claude both work. What do you write about the one you did not take?
3. The same button in six places. Why is `@apply` not the fix, and what is?

► **GO DEEPER** — the reference deck is optional depth, and it is not examined: the full translation table, the three scales, every prefix and variant, `@theme`, arbitrary values, the reset Tailwind brings, `flex-1` measured, `order` and the Tab key, the v3 config against v4 — and every lecture slide the cycles replaced. The next slide lists it.

GO DEEPER

The reference deck for this week — every slide the lecture pointed at, in order.

[1 · The Tailwind catalogue](#) ← spine 27

[3 · Four costs, and you paid all of them](#) ← spine 5

[9 · Ask both of them to do four things](#) ← spine 15

[10 · Because a class can carry a condition. An attribute cannot.](#) ← spine 14

[12 · And it never fights the CSS you do write](#) ← spine 4

[13 · Preflight: the reset you already wrote](#) ← spine 4

[15 · Ask the browser instead](#) ← spine 4

[16 · The whole translation, by property family](#) ← spine 4

[19 · flex-1 is already a zero basis](#) ← spine 9

[21 · The scales are the product. The short names are a side effect.](#) ← spine 4

[22 · The three scales](#) ← spine 4

[23 · One number moves the whole page](#) ← spine 4

[24 · @theme — the syntax](#) ← spine 4

[25 · Square brackets: a test, not a taboo](#) ← spine 4

[26 · Arbitrary values — syntax, and the test](#) ← spine 4

[27 · Week 5's rule, now impossible to get backwards](#) ← spine 9

[28 · Responsive prefixes](#) ← spine 9

[29 · order moves the paint, not the Tab key](#) ← spine 9

[31 · State variants — the ones you reach for](#) ← spine 14

[35 · Both ramps, on screen at once](#) ← spine 14

[37 · What you will see in older tutorials](#) ← spine 19

[38 · The ladder has three rungs, and you are on the second](#) ← spine 22

[39 · And the one place it is right](#) ← spine 22

[40 · Repeated markup — the three rungs](#) ← spine 22

[41 · The five, and how to catch each in your own file](#) ← spine 22

[42 · The five, one line each](#) ← spine 22

[43 · Where to look, in order](#) ← spine 27